

کنترل ربات با ژست یا حرکات دست

— Hand Gesture Robot Control مستندات پروژه

کنترل بی‌سیم یک ربات چهارچرخ (Differential Drive) با استفاده از تشخیص ژست دست از طریق دوربین، پردازش تصویر با MediaPipe در پایتون، و ارسال دستور از طریق Wi-Fi پروتکل UDP به یک برد ESP32 که موتورهای ربات را از طریق درایور L298N کنترل می‌کند.

معرفی و معماری کلی

سیستم از سه بخش اصلی تشکیل شده است:

[تاپ‌دوربین لپ]
-> [MediaPipe تشخیص دست با پایتون]
-> [Wi-Fi روی UDP]
-> [ESP32]
-> [L298N]
-> [موتورهای ربات]

- لایه‌ی تشخیص (Python + MediaPipe): فریم‌های دوربین را می‌خواند، نقاط اسکلت دست ۲۱ نقطه را استخراج می‌کند و ژست‌ها را به صورت اشیاء Command تولید می‌کند.
- لایه‌ی خروجی (Sinks): هر Command هم‌زمان به چند مقصد فرستاده می‌شود: کنسول، فایل لاگ JSON، وب سوکت نمایشگر زنده مرورگر و UDP ارتباط با ربات.
- لایه‌ی دریافت (ESP32): برد به‌طور مداوم روی یک پورت UDP گوش می‌دهد و بسته‌های دریافتی را به دستورات حرکتی نگاشت کرده و پین‌های L298N را کنترل می‌کند.

قطعات مورد نیاز

سخت‌افزار

- برد ESP32 مدل ESP32-WROOM-32U، ۳۲ پین
- درایور موتور L298N
- ۴ عدد موتور + DC شاسی ربات + ۴ عدد چرخ
- منبع تغذیه‌ی اصلی ربات ۲ باتری لیتیوم یون 3.7v سایز ۱۸۶۵۰ + جاباتری دابل برای باتری لیتیوم یون ۷۳/۷ سایز ۱۸۶۵۰ با اتصال سری
- مازول مبدل DC-DC کاهنده LM2596 برای کاهش ولتاژ باتری به ۵ ولت پایدار جهت تغذیه‌ی ESP32

- برد مورد مینی

نرم افزار

- Python نسخه ۱۰.۳ به بالا
- Arduino IDE برای آپلود کد روی ESP32
- کتابخانه‌های پایتون: opencv-python, mediapipe, pyyaml, flask, websockets

نصب و راه اندازی نرم افزار پایتون ۳.

۱ (مخزن پروژه را دانلود کرده و وارد پوشه‌ی آن شوید:

```
cd hand_gesture_control
```

۲ (کتابخانه‌های مورد نیاز را نصب کنید:

```
pip install -r requirements.txt
```

۳ (فایل مدل تشخیص دست (hand_landmarker.task)، حدود ۱۰ مگابایت (باید در پوشه‌ی models/ باشد. در صورت خطای دانلود خودکار مثلاً 403 Forbidden، آن را دستی دانلود کنید:

```
mkdir models
Invoke-WebRequest -Uri "https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task" -OutFile "models\hand_landmarker.task"
```

۴ (برنامه را اجرا کنید:

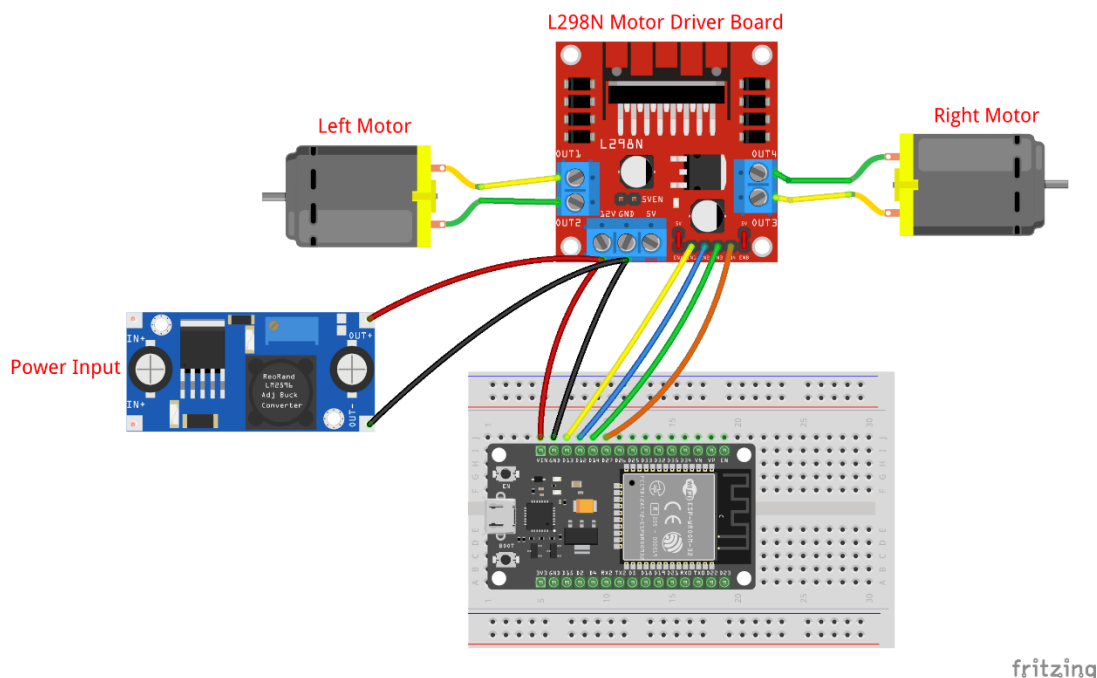
```
python main.py
```

پس از اجرا، یک پنجره‌ی دوربین با اسکلت دست نمایش داده می‌شود و نمایشگر زنده‌ی وب در آدرس <http://127.0.0.1:8000> در دسترس خواهد بود.

سیم‌کشی سخت افزار

اتصال ESP32 به L298N :

پین (GPIO) ESP32	پین روی L298N
۱۳	IN1
۱۲	IN2
۱۳	IN3
۲۷	IN4
VIN ولتاژ مثبت	VCC
GND	GND



شکل: نمای کامل سیم‌کشی ESP32، درایور L298N، ماژول تغذیه LM2596 و موتورهای چپ/راست

نکته: در این پیکربندی، پایه‌های ENA و ENB روی درایور L298N باید مستقیماً به ولتاژ تغذیه یا از طریق جامپر پیش‌فرض روی برد وصل بمانند تا موتورها همیشه با حداکثر سرعت فعال باشند. این کد کنترل سرعت متغیر (PWM) ندارد.

LM2596 (ماژول) تغذیه

در این پروژه کل ربات با ولتاژ ۵ ولت کار می‌کند. برای تأمین این ولتاژ پایدار از ۲ باتری لیتیوم یون که ولتاژ باتری ۳/۷ ولت هست که بصورت سری متصل شده اند از یک ماژول مبدل DC-DC کاهنده از نوع LM2596 استفاده می‌شود. این ماژول ولتاژ ورودی باتری را به‌طور دقیق و قابل‌تنظیم به ۵ ولت کاهش می‌دهد.

- ورودی ماژول LM2596 IN+ و IN- به دو سر باتری اصلی متصل می‌شود.
- خروجی ماژول OUT+ و OUT- باید پیش از اتصال، با مولتی‌متر روی دقیقاً ۵ ولت تنظیم شود و با پیچاندن پتانسیومتر روی ماژول.
- خروجی ۵ ولت ماژول به پین VIN و GND برد ESP32 و همچنین به ورودی تغذیه‌ی منطقی درایور L298N وصل می‌شود.
- استفاده از LM2596 به‌جای رگولاتور خطی، به‌دلیل بازده بالاتر و گرمای کمتر، برای این کاربرد تغذیه از باتری مناسب‌تر است.
- ESP32 از طریق خروجی ۵ ولت ماژول LM2596 یا در زمان برنامه‌ریزی، از طریق کابل Micro Usb تغذیه می‌شود.
- موتورها از مبدل LM2596 از طریق L298N تغذیه می‌شوند.

- زمین (GND) تمام قطعات ESP32، L298N، باتری موتور باید مشترک باشد.

ESP32 کد

ویژگی‌های این کد:

اول از همه کد همراه همین مستند در فولدری به نام wifi قرار گرفته که خب فایل داخل اون با فرمت یا پسوند .ino هست که یعنی باید با نرم افزار آردوینو بازش کنید.

- اتصال با آی‌پی ثابت (Static IP) تا آدرس هرگز توسط روتر تغییر نکند.
- غیرفعال‌سازی حالت خواب Wi-Fi با `WiFi.setSleep(false)` برای جلوگیری از تأخیر یا قطع دریافت بسته‌ها.
- اتصال مجدد خودکار در صورت قطع شدن Wi-Fi.
- گوش دادن روی پورت UDP شماره ۴۲۱۰ و نگاشت کد دریافتی به حرکت مربوطه.

نگاشت ژست‌ها به دستورات

سیستم تشخیص، هر ژست را با یک کد عددی یکتا (`Command.code`) شناسایی می‌کند. جدول زیر نگاشت استفاده‌شده در این پروژه را نشان می‌دهد:

خروجی اعداد	حرکت ربات
9, 10, 11	جلو (Forward)
15, 16	عقب (Backward)
6	توقف (Stop)
0, 1, 2, 3	چپ (Left)
12, 13, 14	راست (Right)

(سمت پایتون) UDP Sink کد

فایل `core/output/udp_sink.py` مسئول ارسال کد دستور به ESP32 است. این فایل شامل یک فیلتر «پایداری» (stability) است که از ارسال کدهای نویزی و لحظه‌ای جلوگیری می‌کند:

```
import socket
from .base_sink import OutputSink
from ..command import Command

class UDPSink(OutputSink):
    def __init__(self, ip="192.168.1.150", port=4210, categories=None, stability_count=2):
        super().__init__(categories)
        self.addr = (ip, port)
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```

self._last_sent_code = None
self._candidate_code = None
self._candidate_count = 0
self._stability_count = stability_count

def emit(self, command: Command):
    if not self.should_emit(command):
        return
    code = command.code
    if code == self._candidate_code:
        self._candidate_count += 1
    else:
        self._candidate_code = code
        self._candidate_count = 1
    if self._candidate_count < self._stability_count:
        return
    if code == self._last_sent_code:
        return
    self._last_sent_code = code
    try:
        self.sock.sendto(str(code).encode("utf-8"), self.addr)
        print(f"[UDPSink] ارسال شد: {code}")
    except OSError as e:
        print(f"[UDPSink] خطا: {e}")

def close(self):
    self.sock.close()

```

اجرای کامل سیستم

- برد ESP32 را روشن کنید و صبر کنید به Wi-Fi متصل شود و قابل بررسی از طریق Serial Monitor با baudrate ۱۱۵۲۰۰.
- مطمئن شوید آی پی داخل main.py تنظیمات UDPSink با آی پی ثابت ESP32 یکسان است.
- برنامه‌ی پایتون را با دستور python main.py اجرا کنید.
- دست خود را جلوی دوربین بگیرید و ژست‌های تعریف شده را انجام دهید.

تنظیمات قابل تغییر

پارامتر	توضیح
ip	آی پی ثابت برد ESP32
port	پورت UDP پیش فرض ۴۲۱۰
categories	فقط دستورات این دسته‌ها فرستاده می‌شود، مثلاً "gesture", "movement"]
stability_count	تعداد فریم متوالی لازم برای پذیرفتن یک کد به عنوان پایدار؛ عدد بالاتر یعنی پایداری بیشتر ولی واکنش کندتر

عیب‌یابی (Troubleshooting)

مشکل	علت احتمالی / راه‌حل
کد چاپ می‌شود ولی به ESP32 نمی‌رسد	آی‌پی نادرست در — main.py از IP ثابت استفاده کنید
فقط چند بسته دریافت و بعد قطع می‌شود	حالت خواب (WiFi.setSleep(false) — Wi-Fi ESP32) اضافه شود
با اینکه دست ثابت است، کد مدام تغییر می‌کند	نویز طبیعی تشخیص‌گر stability_count — را افزایش دهید
هر فریم یک عدد جدید ارسال می‌شود	دسته پر حجم فیلتر نشده categories — را محدود کنید
خطای ModuleNotFoundError	مسیر import اشتباه — مسیر واقعی پوشه را بررسی کنید
خطای دانلود مدل (HTTP 403)	مدل را دستی با Invoke-WebRequest دانلود کنید
ESP32 بعد از اتصال موتور ریست می‌شود	افت ولتاژ — تغذیه‌ی موتور را از ESP32 جدا کنید

توسعه‌های آینده

- افزایش دقت تشخیص دست: تنظیم دقیق‌تر آستانه‌های اطمینان (confidence) مدل MediaPipe، بهبود نورپردازی محیط تست، و افزودن فیلتر هموارسازی (Smoothing) روی نقاط اسکلت دست برای کاهش لرزش و تشخیص اشتباه ژست‌ها.
- افزودن کنترل سرعت متغیر بر اساس شدت حرکت دست با بازگرداندن ENA/ENB و PWM
- اضافه کردن حسگر فاصله برای جلوگیری از برخورد خودکار
- پشتیبانی از چند ربات هم‌زمان با شناسه‌ی دستگاه در پیام UDP